

AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Topic 2 – AI Food Analyzer

AI Food Analyzer Team

Farhad Budagov • Mikayil Hasanov • Nihad Gojayev

Final Project Defense – 19 September 2026

`github.com/budagovfn/ai-food-analyzer`

The problem in one sentence

What the system does

A user uploads a meal photo; we identify the ingredients, look up nutrition facts for each, and return total calories, protein, carbs, and fat, saved to a persistent history.

Inputs

- ▶ Meal photo (JPEG/PNG, ≤ 5 MB)
- ▶ Nothing else – no manual portion entry

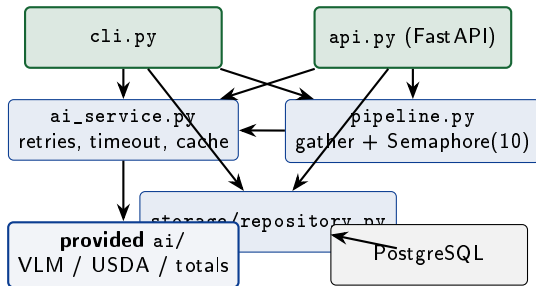
Outputs

- ▶ Per-ingredient kcal/protein/carbs/fat + totals (JSON or table)
- ▶ History record in PostgreSQL

Entry points: `python -m foodanalyzer analyze <path>` (CLI) + `POST /analyze`, `GET /health` (FastAPI)

Architecture

Module map. The boundary between the *provided* ai/ package and *our* SE layer is shown explicitly.



*Every arrow into ai/ passes through ai_service.py – nothing else imports a provider directly.
Swapping Gemini for Anthropic/OpenAI is a .env change, 0 lines of src/ code.*

Three design decisions we can defend

1. `asyncio.gather` + **Semaphore(10)**, not a bare thread pool
`api.py` is already an async FastAPI app; `asyncio` lets a slow lookup batch yield the event loop to other requests, which a blocking `ThreadPoolExecutor.map` inside a route handler would not.
2. **A permanent upload directory**, not `NamedTemporaryFile(delete=True)`
Windows refuses to reopen a file whose handle is still held – caused a `PermissionError` on every single request (Slide 6). Rejected alternative: catching the error and retrying, which papers over a design bug instead of fixing it.
3. **One shared DB connection pool per process**, not one per request
Opening a fresh `asyncpg` pool on every `POST /analyze` risked exhausting Postgres's connection limit under load; a FastAPI lifespan handler now opens it once.

Expect to be asked

“Why not a token-bucket rate limiter instead of a semaphore?” – we chose the simpler bound because our real workload (single-digit ingredients per meal) never approaches

Concurrency: the benchmark

Workload: 10 distinct ingredients (rice, chicken, broccoli, salmon, potato, egg, salad, pasta, tomato, cheese), fresh NutritionService each run so neither side gets a cache hit.

	Sequential	Concurrent (sem=10)
Wall-clock (s)	15.69	3.16
Speedup	1.0×	4.96×
USDA 429s	0	0

Bottleneck named: network round-trip time to USDA per ingredient – the lookup is I/O-bound, not CPU-bound.

What didn't reach 10×: with $N = 10$ = semaphore bound, all lookups start together, so wall-clock is set by the *slowest* single response, not an even 1/10 share – real latency variance means we land at $\sim 5\times$, not $10\times$.

Reproduce with: `python scripts/benchmark_concurrency.py` (needs a real `USDA_API_KEY`).

Numbers must reproduce from the README. Have the exact command line ready.

Robustness: two failure modes, one loud and one silent

Loud: 500 on every request

- ▶ `NamedTemporaryFile(delete=True)` holds its handle; `ai/` reopens the image by path; Windows refuses the second handle.
- ▶ The traceback named it exactly: `PermissionError: [WinError 32]`.
- ▶ Fix: a permanent UUID-named path under `data/uploads/`, unlinked on success *and* failure.

Silent: 200 OK, wrong number

- ▶ `/analyze` reported **1975 kcal** for a 240 g plate — valid schema, correct types, nothing failed.
- ▶ Atwater on that same response gives **459 kcal**. Ratio **4.30**; kJ per kcal is **4.184**.
- ▶ `ai/nutrition.py` drops `unitName`; USDA sends Energy twice and KJ wins. Fixed in *our* layer — §4.8 forbids editing `ai/`.

Why the silent one matters more

Retries, timeouts and graceful degradation all handle calls that *fail*; none engage when a call succeeds with a plausible-looking number. A physical invariant caught it, not another test.

Live demo — from a clean clone

What we will run

The whole system brought up from the repository with a single command, then the same photo through both front doors, showing identical totals.

```
$ docker compose up --build      # API + PostgreSQL 16, one command

$ curl.exe http://localhost:8000/health
$ curl.exe -F "file=@data/rice_chicken_broccoli.png" http://localhost:8000/analyze

$ docker compose run --rm api python -m foodanalyzer analyze data/rice_chicken_broccoli.png
```

- ▶ Same ingredients, same totals, two front doors — both entry points share one business-logic path.
- ▶ The image carries the CLI and the offline demo as well as the server: `docker compose run --rm api python demo_ai.py --offline`.
- ▶ `data/no_meal_blue.png` shows the graceful `meal_recognized=false` branch, not a crash.

Testing

Coverage & shape

- ▶ **93 %** on `src/`, **64 %** with `ai/`; **102** tests, offline
- ▶ `api.py` 90 %, `cli.py` 84 %, `ai_service.py` 97 %; the rest of `src/` 100 %
- ▶ Uncovered remainder: the provided `ai/providers/` adapters
- ▶ Provided `test_ai_smoke.py` untouched, 29 passing

Four tests worth calling out

- ▶ Happy path: all ingredients priced
- ▶ Concurrency: one lookup fails, batch survives
- ▶ Retry-exhausted → `AIServiceError`, never a raw timeout
- ▶ Unit audit: the 1975 kcal payload, wrong before the fix, consistent after

Honesty note

Our timing test used to be vacuous — an autouse fixture disabled `time.sleep` globally, killing the delay it measures. Now fixed, and verified by sabotage: sequential code fails in 2.5 s where it used to pass in 0.07 s.

Limitations & what we'd do next

Where the system is brittle today

- ▶ **Identification is the weakest link.** On `rice_chicken.png` the VLM reported a “grilled beef patty”, and portions come back as round numbers. Our unit audit catches a wrong *unit*, never a wrong *ingredient*.
- ▶ No load test against concurrent *requests* (only concurrent lookups within one request); the likeliest failure on a busy day is Postgres pool exhaustion at `max_size=5`, not the AI layer.
- ▶ Single-provider, no failover — a Gemini outage or an invalid USDA key fails that stage after 3 retries, with no fallback provider.

Given another week we would

1. Write a contract test for the USDA adapter: one recorded payload, replayed offline. It would have caught the kilojoule defect on day one.
2. Run a real load test against the Docker image, then add a second VLM/nutrition provider for failover.

Team contributions

Member	Primary ownership	Commits
Farhad Budagov	Config and storage layers, HTTP API fixes, Docker hardening, unit-audit provider, git/PR workflow, report & slides	33/47 (17/29)
Mikayil Hasanov	AI-service wrapper (retry/timeout/cache), concurrency pipeline, FastAPI endpoint, Dockerfile & compose	9/47 (9/29)
Nihad Gojayev	API and CLI test suites, README, pull-request review and merges	5/47 (3/29)

From `git shortlog -sn` with the repository's `.mailmap`, which collapses the several git identities we each committed under. In italics: excluding merge commits — 16 of the 18 merges on `main` are Farhad's, from running the PR workflow, and count towards a total without being authored code.

AI tool use: Claude reviewed and hardened the Docker PR, fixed two failing tests, found the kilojoule defect on Slide 6 and wrote the provider guarding against it, and drafted this deck and the report from our commit history. Every approval and merge was made by a team member; full disclosure in §9 of the report.

We can all answer:

Pick any file in the repo — whoever owns it will walk you through it.

Questions?

AI Food Analyzer Team

`github.com/budagovfn/ai-food-analyzer`

“We can walk through any file in the repo.”